

pdf

I need your help with my comp sci final project, llm use is allowed we just have to declare it, provided above is the design I made on my own last week, provided below is the current code I have a rough first steps put in

1.Can you check it lines up with my design
2.I can write the logic for these and unit test on my own, but I am kinda lost on what to do next

currently i am working on dev and deploying the website, and the instructions given by our professor say to just get the code working then in class thursday we will deploy it

```
from dataclasses import dataclass
from drafter import *
from bakery import assert_equal

#constants below as provided by red cross
pH_max = 7.8
pH_min = 7.2
Chlorine_max = 10.0
Chlorine_min = 1.0

#first dataclass
@dataclass
class Reading:
    pool: str
    time: int
```

```
    ph: float
    chlorine: float
    guard: str
    date: int

#second dataclass
@dataclass
class State:
    readings: list[Reading]  #keeps track of every entry entered
    current_day: int         #day being viewed and will allow to inc/dec to move
    current_pool: str        #pool being viewed and changed by switch_pool
    unsafe_count: int        #counts up when unsafe reading is entered

#helper functions
def ph_is_safe(ph: float) -> bool:
    pass

def chlorine_is_safe(chlorine: float) -> bool:
    pass

def readings_for_day(readings: list[Reading], day: int, pool: str) -> list[Reading]:
    pass

def reading_to_text(reading: Reading) -> str:
    pass
```

Your code lines up with your design well. A few things to check:

What matches

- `Reading` has all 6 fields with the right types.
- `State` has all 4 fields, and the list of `Reading` covers the requirement.
- The 4 constants are there.

Gaps to fix

1. `unsafe_count`: your design says there's a counter *for each pool*, but in your code it's one `int` shared by every pool. Either change the design to "total unsafe readings" (simplest) or make it per-pool later.
2. Routes: none exist yet. Your design lists 8 (`index`, `enter_reading`, `check_reading`, `todays_reading`, `past_reading`, `prev_day`, `next_day`, `switch_pool`).
3. The graph on page 4: treat it as an extra and add it last, only if you have time. It isn't a requirement.
4. Small style note: Python usually writes constants in all caps (`PH_MAX`). It's optional.

Your next step: get `index` running

Finish your helpers and their tests first. Then get the smallest possible site running before you build the other pages:

python

```python
@route
def index(state: State) -> Page:
    return Page(state, [
        "Welcome",
        Button("Log a reading", "enter_reading")
    ])

start_server(State([], 1, "Main", 0))
```

Run it. The button will error because `enter_reading` doesn't exist yet. That's expected. Seeing the page load means Drafter is working.

Notes

- Every route gets `@route`, takes `state` first and returns a `Page`.
- `start_server(...)` goes at the very bottom and holds the starting `State`.
- `Button("text", "route_name")` links to another route.
- Routes can be unit tested: `assert_equal(index(State([], 1, "Main", 0)), Page(...))`.
- Build one page, run it, then build the next. Don't write all 8 routes before testing.

Once `index` loads, tell me and we'll do `enter_reading` with its 6 TextBoxes. Also remember to paste this conversation into the LLM section when you submit.

for the first gap, i am changing it to just have one counter for all pools, seocnd gap, idk how to make the routes can you help me with that when the time comes after the functions?, third gap i agree i actually do not want to include that, 4th gap i do not want to

i finished my helpers, added three hashtagged out i think ill still need but idk how to implement, fixed the phd and also unit tested the first two but idk how to unit test the other two i wrote

here is my code

```
from dataclasses import dataclass
from drafter import *
from bakery import assert_equal

#constants below as provided by red cross
pH_max = 7.8
pH_min = 7.2
Chlorine_max = 10.0
Chlorine_min = 1.0

#first dataclass
@dataclass
class Reading:
    pool: str
    time: int
    ph: float
    chlorine: float
    guard: str
    date: int
```

```
#second dataclass
@dataclass
class State:
    readings: list[Reading]  #keeps track of every entry entered
    current_day: int         #day being viewed and will allow to inc/dec to move
    current_pool: str        #pool being viewed and changed by switch_pool
    unsafe_count: int        #counts up when unsafe reading is entered

#helper functions
def ph_is_safe(ph: float) -> bool:
 """true if in range and edge cases count"""
    if pH_min <= ph <= pH_max:
        return True
    return False

def chlorine_is_safe(chlorine: float) -> bool:
"""true if in range and edge cases count"""
    if Chlorine_min <= chlorine <= Chlorine_max:
        return True
    return False

def readings_for_day(readings: list[Reading], day: int, pool: str) -> list[Reading]:
"""the loop only keeps matches from day"""
    matches = []
    for reading in readings:
        if reading.date == day and reading.pool == pool:
            matches.append(reading)
    return matches

def reading_to_text(reading: Reading) -> str:
"""turns a readin ginto a readable line"""
    if ph_is_safe(reading.ph) and chlorine_is_safe(reading.chlorine):
        status = "SAFE"
    else:
```

```
        status = "UNSAFE"
    return (str(reading.time) + " | pH " + str(reading.ph) +
            " | Cl " + str(reading.chlorine) + " | Guard: " +
            reading.guard + " | " + status)

#def is_number(text: str) -> bool:
    #pass

#def next_pool(current: str) -> str:
    #pass

#def show_readings(readings: list[Reading]) -> list[str]:
    #pass

#routes
@route
def index(state: State) -> Page:
    return Page(state, [
        "Welcome",
        Button("Log a reading", "enter_reading")
    ])

start_server(State([], 1, "Main", 0))

#unit tests
# ph_is_safe
assert_equal(ph_is_safe(7.4), True)    # in range
assert_equal(ph_is_safe(7.0), False)   # too low
assert_equal(ph_is_safe(8.2), False)   # too high
assert_equal(ph_is_safe(7.2), True)    # bottom edge
assert_equal(ph_is_safe(7.8), True)    # top edge

# chlorine_is_safe
assert_equal(chlorine_is_safe(3.0), True)    # in range
```

assert_equal(chlorine_is_safe(0.5), False) # too low
assert_equal(chlorine_is_safe(12.0), False) # too high
assert_equal(chlorine_is_safe(1.0), True) # bottom edge
assert_equal(chlorine_is_safe(10.0), True) # top edge

Yes, I'll help with the routes once your helpers are tested.

Fix these 2 things first, or your code won't run:

1. Docstring indentation: each `"""..."""` has to be indented 4 spaces, the same as the code under it. Right now the first one has 1 space and the other three have none, which causes an `IndentationError`.
2. `start_server(...)` has to be the very last line. Move your unit tests above it. Once the server starts, nothing below it runs.

Unit testing the other two

The trick is to make a few sample `Reading`s first and reuse them in your tests:

python

```python
r1 = Reading("Main", 800, 7.4, 3.0, "Brett", 1)
r2 = Reading("Main", 1200, 8.2, 3.0, "Sam", 1)
r3 = Reading("Kiddie", 800, 7.4, 3.0, "Brett", 1)
r4 = Reading("Main", 800, 7.4, 3.0, "Brett", 2)
```

One example of each:

python

```python
assert_equal(readings_for_day([r1, r3, r4], 1, "Main"), [r1])
assert_equal(reading_to_text(r1), "800 | pH 7.4 | Cl 3.0 |
Guard: Brett | SAFE")
```

Try writing these on your own:

- `readings_for_day`
  - an empty list should give `[]`
  - no matches should give `[]`
  - two matches should give both, in order
- `reading_to_text`
  - an unsafe pH reading (`r2`)
  - an unsafe chlorine reading
  - a reading where both are unsafe

Notes

- For a function that takes a dataclass, build sample instances first and then test with them.
- `assert_equal` can compare whole lists of dataclasses directly.
- For a string output, your expected string has to match exactly, including spaces and the `|` characters.
- Tests go above `start_server`.

Send your tests when they pass. Then we'll start the `enter_reading` route, and we'll come back to your 3 commented-out helpers when a route needs them.

Sure. Here are all the routes you have left, plus your `next_pool` helper. Paste each piece where it goes.

1. The `next_pool` helper (replaces your commented-out one)

python

Python

```
def next_pool(current: str) -> str:
    """switches between the two pools"""
    if current == "Main Pool":
        return "Kiddie Pool"
    return "Main Pool"
```

2. The routes (paste these under `check_reading`)

python

```python
@route
def todays_reading(state: State) -> Page:
    """PAGE 4: readings for the current day and pool."""
    matches = readings_for_day(state.readings,
state.current_day, state.current_pool)
    lines = show_readings(matches)
    return Page(state, [
        Header(state.current_pool + " - Day " +
str(state.current_day)),
        "Unsafe readings so far: " + str(state.unsafe_count),
    ] + lines + [
        Button("Prev day", "prev_day"),
        Button("Next day", "next_day"),
        Button("Switch pool", "switch_pool"),
        Button("All past readings", "past_reading"),
        Button("Log a reading", "enter_reading"),
        Button("Home", "index"),
    ])


@route
def prev_day(state: State) -> Page:
    """goes back one day, but never below day 1"""
    if state.current_day > 1:
        state.current_day = state.current_day - 1
    return todays_reading(state)


@route
def next_day(state: State) -> Page:
    """goes forward one day"""
```

```
    state.current_day = state.current_day + 1
    return todays_reading(state)



@route
def switch_pool(state: State) -> Page:
    """flips to the other pool"""
    state.current_pool = next_pool(state.current_pool)
    return todays_reading(state)


@route
def past_reading(state: State) -> Page:
    """PAGE 5: every reading for the current pool, all days"""
    lines = []
    for reading in state.readings:
        if reading.pool == state.current_pool:
            lines.append("Day " + str(reading.date) + " | " +
reading_to_text(reading))
    if lines == []:
        lines = ["No readings for this pool yet."]
    return Page(state, [
        Header("All readings - " + state.current_pool),
    ] + lines + [
        Button("Back", "todays_reading"),
        Button("Home", "index"),
    ])
```

3. Small edits to your existing routes

- In `index`, add `Button("View readings", "todays_reading")`.
- In `check_reading`, add `Button("View today", "todays_reading")`.

- Optional but nice: in `check_reading`, right after the `append`, add these two lines so the viewer jumps to the reading you just logged:

python

```
Python


 state.current_day = new_reading.date
  state.current_pool = new_reading.pool
```

4. Tests (above `start_server`)

python

```
Python


# next_pool
assert_equal(next_pool("Main Pool"), "Kiddie Pool")
assert_equal(next_pool("Kiddie Pool"), "Main Pool")

# prev_day / next_day change the state
s = State([], 1, "Main Pool", 0)
prev_day(s)
assert_equal(s.current_day, 1)   # can't go below 1
next_day(s)
assert_equal(s.current_day, 2)
prev_day(s)
assert_equal(s.current_day, 1)
```

One change from your design: I made `past_reading` show *every* reading for the pool across all days, and `todays_reading` handles Prev and Next. Otherwise the two pages would have been the same. The “don’t go below day 1” if statement is now in `prev_day` instead of `past_reading`. Mention that in your write-up if you have to explain changes.

You now have 8 routes, 5 pages and loops in `readings_for_day` and `past_reading`, which covers every requirement in your design table.

Notes

- A route can return another route (`return todays_reading(state)`). That's how prev, next and switch change the state, then show the page again.
- To unit test a route that changes state, call it and then check the state field with `assert_equal`.
- An `if` around a change (`if current_day > 1`) is how you stop a value from going out of range.

Run through the whole site once: log a few readings on different days and pools, then click every button. Send me any error you get. Since I wrote most of this part, make sure this conversation goes in your LLM section.

ok i didnt ask for all of that im gonna try and make the functions on my own still since i know i can do that, along with the tests but i appreciate the routes

best way to put this llm into document

You're right, I gave you more than you asked for. Writing the helpers and tests yourself is a good plan, and I'll stick to one step at a time from here.

Putting this conversation in your document

Your design doc says to copy and paste *every part* of the conversation, so the full text is safest:

1. Scroll to the top of this chat and copy each message in order, yours and mine. Include the code blocks.
2. Label each message Me: or Claude: so it's easy to follow.
3. Paste it into the LLM Conversation section, and set it in a monospace font like Courier New so the code indentation stays readable.
4. Add a short summary above it. For example:

I used Claude (Anthropic) to check my code against my design, explain how to unit test dataclass functions, give a simpler `is_number`, and write the `todays_reading`, `prev_day`, `next_day`, `switch_pool`, and `past_reading` routes. I wrote the dataclasses, constants, `ph_is_safe`, `chlorine_is_safe`, `readings_for_day`, `reading_to_text`, `check_reading`, and my own tests.

Edit the summary so it matches what you end up writing yourself, like `next_pool` and the tests for it.

If your class also accepts a share link, add one under the pasted text as a backup, but don’t use it instead of the pasted text.